

PART V: DEPLOYMENT - CONTAINERIZED ARCHITECTURE

Goal: formalize the distributed system environment using Docker, moving from ad-hoc process management to an orchestrated microservices architecture.

1. Architectural Justification

Instead of VMs, we utilize **OS-level virtualization (Containers)**. This allows us to:

1. **Deterministic Environment:** Guarantee that the memory layout and network stack are identical for all students, ensuring exploits (like Buffer Overflows) behave consistently.
2. **Resource Quotas:** We will artificially constrain the Coordinator's CPU resources. This is critical for the **DoS simulation**, as it prevents your powerful host machine from masking the inefficiency of the naive server implementation.
3. **Network Isolation:** We will define a software-defined bridge network (`172.25.0.0/16`) to isolate experiment traffic from the host.

2. Infrastructure Setup

Step 1: Project Structure Organize your workspace to separate source code from deployment logic.

```
/lab3_dist_systems
├── src/
│   ├── coordinator_secure.c
│   ├── worker_secure.c
│   ├── client.c
│   ├── dos_flooder.c (see Part VI)
│   └── common.h
├── Dockerfile
└── docker-compose.yml
```

Step 2: The Multi-Stage Dockerfile Create a **Dockerfile** that uses a **multi-stage build**. This is an industry standard to keep runtime images small and secure. We compile in a heavy "builder" stage and copy only the binaries to a lightweight "runtime" stage.

```
# --- Stage 1: Builder ---
FROM gcc:12 AS builder
WORKDIR /build
```

```

# Copy source code
COPY src/ .

# Compile binaries with debug symbols (-g) and pthread support
# We compile specific targets to avoid rebuilding everything unnecessarily
RUN gcc coordinator_secure.c -o coord -pthread -g
RUN gcc worker_secure.c -o worker -pthread -g
RUN gcc client.c -o client -g
RUN gcc dos_flooder.c -o flooder -pthread -g

# --- Stage 2: Runtime Environment ---
# We use a standard Debian-slim image for a realistic server environment
FROM debian:bookworm-slim

# Install networking tools for "Blue Team" analysis
# netcat: for manual socket interaction
# tcpdump: for packet capture
# iproute2: for 'ss' and 'ip' commands
# procps: for 'ps' and 'top'
RUN apt-get update && apt-get install -y \
    netcat-openbsd \
    tcpdump \
    iproute2 \
    procps \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Copy only the compiled binaries from the builder stage
COPY --from=builder /build/coord .
COPY --from=builder /build/worker .
COPY --from=builder /build/client .
COPY --from=builder /build/flooder .

# Default entrypoint (overridden by Compose)
CMD ["/coord"]

```

Step 3: Orchestration (docker-compose.yml) Create `docker-compose.yml`. Note the explicit **subnet configuration** and **resource limits**.

```

version: '3.8'
services:

```

```

coordinator:
  build: .
  container_name: coordinator
  command: ./coord
  ports:
    - "9000:9000"
  networks:
    backend_net:
      ipv4_address: 172.25.0.10 # Static IP for legacy clients
  deploy:
    resources:
      limits:
        cpus: '0.5'
        memory: 256M
  sysctls:
    - net.ipv4.tcp_syncookies=1 # Mitigation for SYN Flood

worker:
  build: .
  command: ./worker
  deploy:
    replicas: 3
  networks:
    - backend_net
  depends_on:
    - coordinator

attacker:
  build: .
  container_name: attacker
  entrypoint: ["tail", "-f", "/dev/null"]
  networks:
    - backend_net

# DEFINE THE NETWORK
networks:
  backend_net:
    driver: bridge
    ipam:
      config:
        - subnet: 172.25.0.0/16

```

3. Code Adaptation

You must update the addressing in `src/common.h` to match the static IP defined in the Compose file.

```
// src/common.h
// #define SERVER_IP "127.0.0.1"    <-- Legacy
#define SERVER_IP "172.25.0.10"    // <-- Docker Static IP
```

Workflow:

1. **Build:** `docker compose build`
2. **Deploy:** `docker compose up -d`
3. **Verify:** `docker compose ps` (Check that 5 containers are Up).

4. Network Observability & Protocol Analysis

Unlike VirtualBox Host-Only networks (where the traffic passes through a virtual adapter visible to the host OS), Docker containers on non-Linux systems (Windows/Mac) run inside a lightweight utility VM. This makes "sniffing" the traffic from your Host Wireshark slightly more complex.

We will use two methods. Choose the one suitable for your OS.

Method A: The "Sidecar" Capture (Universal - Recommended for Windows/Mac) Since we cannot easily bridge the Host Wireshark to the internal Docker network on Windows/Mac, we will use `tcpdump` *inside* the Coordinator container to capture traffic to a file, and then analyze it on the host.

Start the Capture: Exec into the running coordinator and start a background capture: `docker exec -d coordinator tcpdump -i eth0 port 9000 -w /app/capture.pcap:`

- `-i eth0`: The container's internal interface.
- `-w ...`: Write to a file instead of printing to stdout.

Generate Traffic: Run your `client` or `worker` interactions.

- *Example:* `docker exec attacker ./client "Packet Capture Test"`

Stop & Extract: Since `tcpdump` is running in the background, we simply copy the file out (the process will die when the container stops, or you can kill it): `docker cp coordinator:/app/capture.pcap ./trace.pcap`

Analysis: Open `trace.pcap` in Wireshark on your host machine.

Method B: Direct Bridge Analysis (Linux Native Only) If you are running Docker natively on Linux, the Docker network appears as a real bridge interface on your host.

Identify the Interface: `ip addr | grep br-`

1. Look for the interface associated with `172.25.x.x`. It will likely be named something like `br-0a1b2c3d4e5f`.
2. **Live Capture:** Open Wireshark, select that bridge interface, and apply the display filter: `tcp.port == 9000`

Analysis Objectives (What to look for):

Once you have the packets in Wireshark (via Method A or B), locate the following frames to verify your binary protocol implementation:

1. **The Handshake (Authentication):**
 - Find the `MSG_WORKER_READY` packet (Type 2).
 - Immediately following it, look for the **Server Challenge** (4 bytes, the `Nonce`).
 - Look for the **Worker Response** (4 bytes, `Nonce ^ Secret`).
 - If you see the Nonce and the Response in plaintext, what does this imply about Man-in-the-Middle (MitM) attacks? (Answer: The protocol is vulnerable to replay attacks or offline derivation of the key if many pairs are captured).
2. **The Payload (Buffer Overflow Context):**
 - Find a `MSG_TASK_SUBMIT` packet.
 - Inspect the "Data" pane in Wireshark. You will see the raw struct bytes.
 - Verify the byte alignment. Is the compiler padding the struct? (This is why `__attribute__((packed))` is often discussed in network programming).
3. **The DoS Artifacts (During Part VI):**
 - If you capture during the DoS attack, you should see thousands of **SYN** packets from `172.25.0.20` followed by **SYN**, **ACK** from the Coordinator, but *missing* the final **ACK** (if using a SYN flood tool) or simply fully established connections that transmit garbage data.

Attack simulation:

1. Exec into the `attacker` container and run a legitimate client: `docker exec -it attacker ./client "Baseline Task"`
2. Inside the attacker container, run the flooder tool: `docker exec -d attacker ./flooder`
 - a. (The `-d` runs the exec command in the background).

3. `docker exec -it attacker ./client "Critical Payload"`
4. `docker exec coordinator netstat -ant | grep :9000 | grep ESTABLISHED | wc -l`
5. `docker exec coordinator ss -lnt`

Advanced Mitigation (Blue Team Implementation)

For the mitigation phase (Part VI, Extension B), you are expected to implement application-layer defense. However, a robust defense requires **Kernel Tuning** as well.

Task: Modify the `docker-compose.yml` to harden the kernel parameters for the Coordinator.

```
coordinator:
  # ... previous settings ...
  sysctls:
    - net.core.somaxconn=1024    # Increase max backlog queue
    - net.ipv4.tcp_max_syn_backlog=2048
```

Note: This strictly buys you time; it does not solve the root cause. The true solution requires the Rate Limiting logic implemented in C (Extension B).

Cleanup & Reset

Between experiments, you must fully tear down the stack to clear the orchestrator state:

```
docker compose down
```